

SIA - Optimización (extras)

A dark blue diagonal gradient bar that starts from the bottom left and extends towards the top right, covering the lower half of the slide.

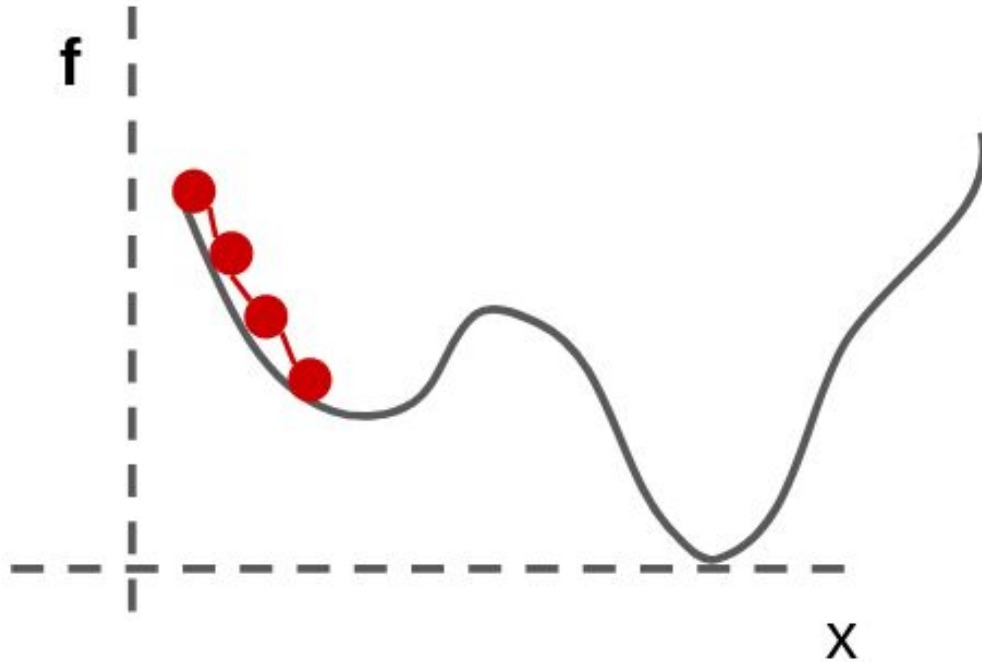
Aprendizaje en redes neuronales

- El método del gradiente descendente nos da una dirección en la cual lograremos navegar esa superficie bajando cada vez el costo.
- Esa dirección es ΔW

Aprendizaje en redes neuronales

- El problema que tiene el método del gradiente descendente es que la información que dispone es local.
- Esto es, porque la dirección de descenso es, justamente, la derivada de $E(W)$ respecto a cada uno de los pesos de las conexiones en el punto obtenido hasta ese momento.

Aprendizaje en redes neuronales



momentum

$$\Delta w_{ij}(t + 1) = -\eta \frac{\partial E}{\partial w_{ij}} + \alpha \Delta w_{ij}(t)$$

donde α está entre 0 y 1. Valores comunes para α son 0.8 o 0.9.

- Si la función $E()$ está en una región plana, $\Delta w_{ij}(t)$ será similar a $\frac{\partial E}{\partial w_{ij}}$, y por lo tanto $\Delta w_{ij}(t + 1)$ será un incremento en módulo de $\frac{\partial E}{\partial w_{ij}}$.
- Si la función $E()$ está en un valle o en una región oscilatoria, $\Delta w_{ij}(t + 1)$ tenderá a compensarse para evitar oscilaciones.

eta adaptativo

η Adaptativo

Si, de forma consistente, $E()$ decrece en un momento del aprendizaje (por ejemplo, durante k iteraciones seguidas), quizás sería interesante incrementar η .

Por otro lado, si $E()$ comienza a incrementarse en un momento del aprendizaje (por ejemplo, durante k' iteraciones seguidas), quizás sería interesante decrementar η para evitar divergencia.

eta adaptativo

η Adaptativo

Esto se puede formalizar adaptando el valor de η según:

$$\Delta\eta = \begin{cases} +a & \text{si } \Delta E < 0 \text{ en forma consistente} \\ -b\eta & \text{si } \Delta E > 0 \text{ empieza a ser consistente} \\ 0 & \text{en otro caso} \end{cases}$$

* 'en forma consistente' se puede parametrizar de muchas formas.
Por ejemplo, si en K épocas el error varía menos de P%

RMSProp

$$g_t = \frac{\partial E}{\partial w_{ij}}$$

$$S_t = \gamma S_{t-1} + (1 - \gamma) g_t^2$$

g_t^2 es el cuadrado aplicado miembro a miembro en el vector g_t

$$\Delta w_{ij} = -\frac{\eta}{\sqrt{S_t + \epsilon}} * g_t$$

ϵ es un parámetro para evitar divisiones por 0 (en las primeras iteraciones)

RMSProp – Intuición

$$g_t = \frac{\partial E}{\partial w_{ij}}$$

$$S_t = \gamma S_{t-1} + (1 - \gamma) g_t^2$$

$$\Delta w_{ij} = -\frac{\eta}{\sqrt{S_t + \epsilon}} * g_t$$

Se ajusta el learning rate de acuerdo a la root mean square (RMS).

Alta RMS: learning rate mas bajo
'amortigua' los cambios.
Reduce 'oscilaciones'

Baja RMS: LR mas alto evita
mínimos locales, aprende mas
rápido

RMSProp – Intuición

Promedio? Donde?

$$\begin{aligned}\mathbf{s}_t &= (1 - \gamma)\mathbf{g}_t^2 + \gamma\mathbf{s}_{t-1} \\ &= (1 - \gamma) (\mathbf{g}_t^2 + \gamma\mathbf{g}_{t-1}^2 + \gamma^2\mathbf{g}_{t-2}^2 + \dots,).\end{aligned}$$

Al desarrollar varias iteraciones, toma la forma de un promedio.
Los términos mas recientes tienen mas peso que los viejos.

Adam

- Presentado en el paper *Adam: A method for stochastic optimization* (Kingma, Ba. 2015)
- Combinación de las ideas de RMSProp y Momentum
- Muy usado en la práctica

Adam

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

Adam

$$\begin{aligned}m_t &\leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \\v_t &\leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \\\hat{m}_t &\leftarrow m_t / (1 - \beta_1^t) \\\hat{v}_t &\leftarrow v_t / (1 - \beta_2^t) \\\theta_t &\leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)\end{aligned}$$

Los términos bias-corrected son necesarios para compensar el bias generado por los beta en las primeras iteraciones (ya que m_t y v_t se inicializan en 0)

Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .